专业：电子信息工程
姓名：程梓洋
学号：3220104081
日期：2024/12/13
地点：应电楼 303

浙江大学实验报告

课程名称：计算机网络与通信 指导老师：吴建德 成绩：

实验名称：串口(UART)通信与数据链路层协议设计 实验类型： 同组学生姓名：王浩然

一、实验目的和要求　　二、实验内容和原理

三、主要仪器设备　　四、操作方法和实验步骤

五、实验数据记录和处理　　六、实验结果与分析

七、讨论、心得

一、实验目的和要求

1.掌握数据链路层的设计和编程，深入理解计算机网络系统分层次设计的优点；

2.练习使用 STM32F4 进行 UART 通信；

3.对于计算机网络分层结构有直观的体验和感受；

4.加强单片机的编程能力。

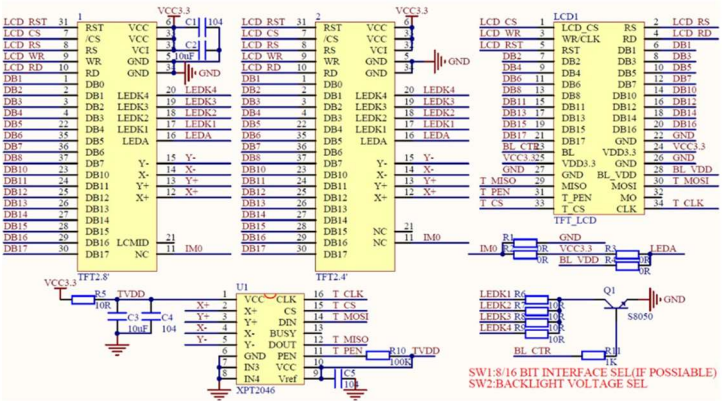
二、实验内容和原理

1.TFTLCD 驱动原理

TFTLCD 即薄膜晶体管液晶显示器。它与无源 TN-LCD、STN-LCD 的简单矩阵不同，它在液晶显示屏的每一个象素上都设置有一个薄膜晶体管(TFT)，可有效地克服非选通时的串扰，使显示液晶屏的静态特性与扫描线数无关，因此大大提高了图像质量。

TFTLCD 具有：亮度好、对比度高、层次感强、颜色鲜艳等特点。是目前最主流的 LCD 显示器。广泛应用于电视、手机、电脑、平板等各种电子产品。

TFTLCD 模块特点：240*320 分辨率，16 位真彩显示(65536 色)，自带电阻触摸屏，自带背光电路。



TFTLCD 模块原理图

2. FSMC 简介

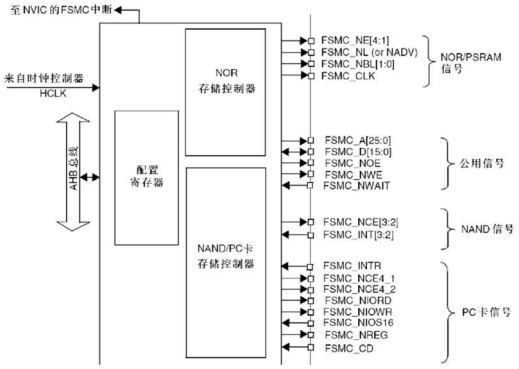
FSMC，即灵活的静态存储控制器，能够与同步或异步存储器和 16 位 PC 存储器卡连接，STM32 的

FSMC 接口支持包括 SRAM、NAND FLASH、NOR FLASH 和 PSRAM 等存储器。

FSMC 驱动外部 SRAM 时，外部 SRAM 的控制一般有：地址线(如 A0~A25)、数据线(如 D0~D15)、写信号(WE，即 WR)、读信号(OE，即 RD)、片选信号(CS)，如果 SRAM 支持字节控制，那么还有 UB/LB 信号。

而 TFTLCD 的信号我们在前面介绍过，包括：RS、D0~D15、WR、RD、CS、RST 和 BL 等，其中真正在操作 LCD 的时候需要用到的就只有：RS、D0~D15、WR、RD 和 CS。其操作时序和 SRAM 的控制完全类似，唯一不同就是 TFTLCD 有 RS 信号，但是没有地址信号。

TFTLCD 通过 RS 信号来决定传送的数据是数据还是命令，本质上可以理解为一个地址信号，比如我们把 RS 接在 A0 上面，那么当 FSMC 控制器写地址 0 的时候，会使得 A0 变为 0，对 TFTLCD 来说，就是写命令。而 FSMC 写地址 1 的时候，A0 将会变为 1，对 TFTLCD 来说，就是写数据了。这样，就把数据和命令区分开了，他们其实就是对应 SRAM 操作的两个连续地址。当然 RS 也可以接在其他地址线上，战舰 V3 和精英板开发板都是把 RS 连接在 A10 上面，而探索者 STM32F4 把 RS 接在 A6 上面。因此，可以把 TFTLCD 当成一个 SRAM 来用，只不过这个 SRAM 有 2 个地址，这就是 FSMC 可以驱动 LCD 原理。



3. 通信层次的划分：2 层

文件传输层	数据链路层
定义所传输的文件格式，包括文名，文件长度，校验方式等	将文件划分为多个数据帧，并定义数据帧格式

(1)文件传输层格式定义

	文件类型	文件名长度	文件名	点阵数据	求和校验
长度/byte	1	1	N	200*240*2	2
注释	标志(AA)	N	字符	16 位像素表示，高字节在前	按字节求和(高字节在前)

(2)数据链路层帧格式定义

数据帧格式：

	Flag	Source_address	Dest_address	Command	Frame Number
长度/byte	1	1	1	1	1
注释	标志(7E)	源地址	目标地址	命令	帧序号

Length	Data	SUMH	SUML
2	N	1	1
数据长度(N)	数据	校验高字节	校验低字节

应答帧格式：

	Flag	Source_address	Dest_address	Command	Serial
长度/byte					
注释	标志(7E)	源地址	目标地址	命令	帧序号

Length	Data	SUMH	SUML
2	0	1	1
长度(N)	数据	校验高	校验低

(3)通信方式：

停止等待 ARQ(PC 主动发命令帧，Cortex 返回确认帧) Cortex 接收命令帧，校验正确后回送确认帧：

1、源地址和目标地址交换；

2、确认帧序号等于命令帧序号；

3、Command 值不变。

(4)如何判断数据帧的完整性(即判断一串数据是否属于同一帧) 上位机发送数据帧要求：

1、一帧中的每个字节间隔小于 Tx(2ms)；

2、两个帧之间的间隔大于 Tx(2ms)；

3、Cortex 每接收到一个字节后都到重置超时定时器。超时计数器溢出，则表示一帧接收完成。

(5)编程方法

1、编写串口中断程序，将完整接收的帧存放在一个数组中；

2、定义一个更大的数组 FileBuff，将接收到的帧数据按顺序保存到该数组；

3、文件全部接收完成后，再调用显示函数将数据在 LCD 上显示。

三、主要仪器设备

STM32F407 单片机，PC 机，Cube MX 软件，keil 软件。

四、操作方法与实验步骤

系统包括一台 PC 和一块 Cortex 开发板。

在 PC 端，上位机输入图片文件，通过 UART 将图片传输到 STM32F407(波特率 115200)，STM32F407

接收后在 LCD 上显示图片。上位机先将输入图片转换为 200*240 的点阵数据，点阵采用 RGB565 格式，即每个点用一个 16 位数表示。再将点阵数据分成多个分组发送给 Cortex 开发板。本次实验提供上位机软件，要求编写 STM32F407 程序。

五、实验数据记录和处理

Main.h 定义结构体：

```
* Private typedef ---        -*/
/* USER CODE BEGIN Private defines */
typedef struct         //
{
uint8_t Flag;
uint8_t SourceAddr;
uint8_t DestAddr;
uint8_t Command;
uint8_t SerialN;
uint8_t LengthH;
uint8_t LengthL;
uint8_t Data[1000];
uint8_t SumH;
uint8_t SumL;
}FrameDef;
typedef struct
{
uint8_t Status;            //当前状态
uint8_t Serial;            //当前序号
uint8_t* sp_FileData;
}ProtocolStatusDef;
/* USER CODE END Private defines */
/* USER CODE END PTD */
```

Main.c 中代码：

```
/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */
FrameDef ReceivedFrame;    //帧结构体，用于保存接收桢
FrameDef SendingFrame;     //帧结构体，用于存放待发送数据帧或应答帧
```

```
uint8_t RFrameFlag;      //接收到一帧标志；=1 表示接收完成，会在函数 CheckFrameSum 中置位，此函数用
                         //于接到桢的校验
ProtocolStatusDef ProtocolStatus;
uint8_t rBuff[1100],SendBuff[100];
uint16_t rLength,Send_data_length;
uint32_t FileLength=0,FilePointer=0;   //FilePointer 用于将接到的数据帧中的数 Filebuff
uint8_t FileRcvCompleteFlag=0;  //文件接收完成标志；=1 接收完成
uint8_t LastFrameSn=255;   //上次接收帧序号，上位机发送桢的顺序为 0~15 循环
uint8_t FileBuff[100000];     //用于给 LCD 传输数据的数组
volatile uint8_t ReceiveState=0, SerialN=0;
volatile uint16_t ptReceive=0;
uint16_t i,j;
/* USER CODE END PTD */

/* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
        if(RFrameFlag == 1)            //如果串口接收到一帧数据
        {
            CheckRcvFrame();     //拼装接收帧
            SendFrame();             //发送应答帧
            RFrameFlag = 0;
        }
        if (FileRcvCompleteFlag==1)   //如果文件接收完毕{
        {
            Show_bmp();                    //显示图片
            FileRcvCompleteFlag=0;
        }
  }
  /* USER CODE END 3 */
void Show_bmp(void)     //显示图片，图片存放在 FileBuff 数组
{
```

```
    uint16_t i,j,color;
    uint32_t n;
    n=FileBuff[1]+1;
    for (i=0;i<240;i++)
    {
        for (j=0;j<200;j++)
        {
            color=FileBuff[n]*256+FileBuff[n+1];
            LCD_SetCursor(i,j);
            LCD_WriteRAM_Prepare();
            TFT_LCD->LCD_RAM=color;
            n++;
            n++;
        }
    }
}

void SendFrame(void)     //发送帧
{
    uint8_t i=0;
    uint16_t sum;
    SendBuff[i++]=0x7e;
    SendBuff[i++]=SendingFrame.SourceAddr;
    SendBuff[i++]=SendingFrame.DestAddr;
    SendBuff[i++]=SendingFrame.Command;
    SendBuff[i++]=SendingFrame.SerialN;
    SendBuff[i++]=SendingFrame.LengthH;
    SendBuff[i++]=SendingFrame.LengthL;
    SendBuff[i++]=SendingFrame.SumH;
    SendBuff[i]=SendingFrame.SumL;
    Send_data_length=9;
    HAL_UART_Transmit(&huart1,SendBuff,Send_data_length,0xffff);
}

void SetAckFrame(void)     //准备发送
```

```
{
    uint16_t sum;
    SendingFrame.Flag=0x7e;
    SendingFrame.SourceAddr=ReceivedFrame.DestAddr;
    SendingFrame.DestAddr=ReceivedFrame.SourceAddr;
    SendingFrame.Command=ReceivedFrame.Command;
    SendingFrame.SerialN=ReceivedFrame.SerialN;
    SendingFrame.LengthH=0;
    SendingFrame.LengthL=0;
    SendingFrame.Data[0]=1;
    sum=0x7e+ReceivedFrame.DestAddr+ReceivedFrame.SourceAddr+ReceivedFrame.Command+ReceivedFr
ame.SerialN;
    SendingFrame.SumH =sum>>8;
    SendingFrame.SumL=sum&0xff;
}
```

stm32f407xx_it.c 中代码：

```
void TIM1_UP_TIM10_IRQHandler(void)          //定时器程序
{
  /* USER CODE BEGIN TIM1_UP_TIM10_IRQn 0 */

  /* USER CODE END TIM1_UP_TIM10_IRQn 0 */
  HAL_TIM_IRQHandler(&htim1);
  /* USER CODE BEGIN TIM1_UP_TIM10_IRQn 1 */
    ReceiveState=0;
    TIM1->SR=0;
  /* USER CODE END TIM1_UP_TIM10_IRQn 1 */
}

void USART1_IRQHandler(void)         //中断程序
{
  /* USER CODE BEGIN USART1_IRQn 0 */
    uint32_t temp32;
    uint8_t temp;
  /* USER CODE END USART1_IRQn 0 */
```

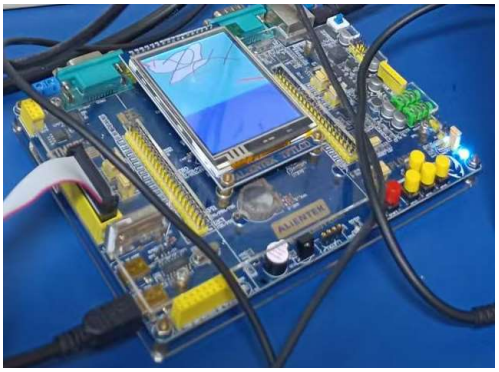
```
HAL_UART_IRQHandler(&huart1);
/* USER CODE BEGIN USART1_IRQn 1 */
   ResetTimer();
   temp32=USART1->SR;
   if((temp32&0x20) != RESET)
   {
        temp = USART1->DR;
        switch (ReceiveState)
        {
             case 0x00:
                  if (temp==0x7e) ReceiveState++;
                  else ReceiveState=255;
             break;
             case 0x01:
                  ReceivedFrame.SourceAddr=temp;
                  ReceiveState++;
             break;
             case 0x02:
                  ReceivedFrame.DestAddr=temp;
                  ReceiveState++;
             break;
             case 0x03:
                  ReceivedFrame.Command=temp;
                  ReceiveState++;
             break;
             case 0x04:
                  ReceivedFrame.SerialN =temp;
                  ReceiveState++;
             break;
             case 0x05:
                  ReceivedFrame.LengthH =temp;
                  ReceiveState++;
             break;
             case 0x06:
                  ReceivedFrame.LengthL =temp;
```

```
                DataNum=ReceivedFrame.LengthH*256+temp;
                ReceiveState++;
                ptReceive=0;
                if (DataNum==0) ReceiveState=8;
                if (DataNum>1000) ReceiveState=255;
            break;
            case 0x07:
                ReceivedFrame.Data[ptReceive]=temp;
                ptReceive++;
                if (ptReceive >= DataNum)      ReceiveState++;
            break;
            case 0x08:
                ReceivedFrame.SumH=temp;
                ReceiveState++;
            break;
            case 0x09:
                ReceivedFrame.SumL=temp;
                CheckFrameSum();
                if (RFrameFlag==1) ReceiveState=0; else ReceiveState=255;
            break;
            case 255:
            break;
        }
    }
  /* USER CODE END USART1_IRQn 1 */
}

void ResetTimer(void)
{
    TIM1->SR=0;
    HAL_TIM_Base_Start(&htim1);
    HAL_TIM_Base_Start_IT(&htim1);
    TIM1->CNT=0;
    TIM1->DIER=0x4;
}
```

```
void CheckFrameSum(void)      //校验程序
{
    uint16_t    sum=0x7e;
    uint16_t    i;
    sum+=(uint16_t)(ReceivedFrame.SourceAddr);
    sum+=(uint16_t)(ReceivedFrame.DestAddr);
    sum+=(uint16_t)(ReceivedFrame.Command);
    sum+=(uint16_t)(ReceivedFrame.SerialN);
    sum+=(uint16_t)(ReceivedFrame.LengthH);
    sum+=(uint16_t)(ReceivedFrame.LengthL);
    for (i=0;i<ReceivedFrame.LengthH*256+ReceivedFrame.LengthL;i++)
    {
        sum+=(uint16_t)(ReceivedFrame.Data[i]);
    }
    if (sum==(uint16_t)(ReceivedFrame.SumH*256+(uint16_t)(ReceivedFrame.SumL)))
        RFrameFlag=1;
}
```

六、实验结果与分析



上述代码烧录后，上位机发送的图片文件可以正确显示在 LCD 屏幕上。在传输过程中插入序号错误、长度错误或校验和错误的数据帧，引起上位机重传。重传完成后，图片仍能正确显示，说明本实验中的程序完整地完成了数据传输、差错检测功能，符合预期要求。

七、讨论、心得

1.本实验中，如何保证数据帧的完整性。此方法与字节填充方法相比，有什么优点或缺点，分别适用于什么场合？

(1)在本实验中，程序接收到 0x7E 后，读取固定长度的帧头，然后读取数据，最后用两个字节的校验和 检

验数据完整性。同时，该程序还通过定时器的运用判断一串数据是否属于同一帧。在上位机发送数据帧 时，要求一帧中每个字节的间隔小于 Tx(2ms)，而两个帧之间的间隔大于 Tx(2ms)。每当 Cortex 接收到一个字节后，超时定时器会被重置。如果定时器溢出，则一帧接收完成。

(2)优点：

简单有效：通过校验和，可以保证数据帧的完整性。

灵活：可以根据协议的需求调整应答帧中的字段。

缺点：

仅能检测简单的错误：校验和对于检测复杂的错误(如多个字节错误)可能不如 CRC 等方法有效，如果有多位错误，校验和可能无法检测到。

效率问题：对于数据量较大的帧，如果需要频繁计算校验和，可能会影响性能。

(3)字节填充方法主要用于处理数据帧长度对齐的需求，而校验和则用于确保数据在传输过程中尽量没有损坏或丢失。如果数据传输环境较好且对数据准确性要求不高，字节填充方法可能就足够。如果需要确保高可靠性和数据的准确性，使用校验和方法更为合适。

2.如可以重新修改数据链路层协议，有哪些方法可以加快图片下载速度？

①提高波特率；②在一定范围内缩短分帧时间；③改善检验错误的方式；④本实验中可以减去每一帧的地址字节，使用点对点线路；⑤对图片采用效率更高的编码方式。

3.本实验中，为什么不能显示 240*320 的图片？如必须显示 240*320 的图片，程序应怎样处理？

在本次实验中，为了显示 200*240 的图片，通过定义了一个 96000 大小的数组来存储数据。若要显示更大的图片，则需要更大的数组，而单片机中的 RAM 有限，显示 240*320 的图片显然超出了 RAM 的内存，故不能显示。

解决方法：我们可以对图片进行分段传送，分段显示，先显示图片的上半部分，然后显示图片的下半部分。